# POLYMATH-ATLAS
# OSA-4.0

*FULL UNIFIED ARCHITECTURAL CODEX*

*Полный проектный корпус: архитектура, OSA, Semantic Kernel, законы, Torture Set, исправления, rejected alternatives, pipeline и границы формальной доказанности. Объединение Master Dossier + Architectural Codex.*

*REFERENCE DOSSIER • 08 AUGUST 2026*

*РЕМАРКА: СТАТУС: Meta-Schema НЕ заморожена и НЕ генерируется до завершения Kernel v0.3.1/v0.4 → Lawbook → Model Check.*

# 0. EXECUTIVE STATUS

| Контур | Статус | Комментарий |
|---|---|---|
| **OSA-3.x** | REFERENCE / LEGACY | Концептуальный и прототипный слой; его дефекты инициировали переход к Formal Core. |
| **OSA-4.0 Kernel v0.1** | SUPERSEDED | Онтология, Type Algebra, Predicate Logic, State, Transition. |
| **Kernel v0.2** | SUPERSEDED | Lifecycle Algebra, Effects, Liveness, Laws. |
| **Kernel v0.3** | SUPERSEDED | Predicate Liveness, Epistemic Join, Verdict Algebra, Lexicographic Repair. |
| **T01–T15** | AUDIT / NOT PROOF | Набор полезен как контрпримерная проверка, но 15/15 не является доказательством soundness. |
| **Kernel v0.3.1 / v0.4** | CURRENT WORKING BASE | WP, полная Join Algebra, ABSENT semantics, recreation/resurrection, authorization, invalidation. |
| **Meta-Schema** | FROZEN / DEFERRED | Не строить до замыкания математической семантики. |

*РЕМАРКА: Нельзя честно объявлять Kernel sound только потому, что заранее придуманные тесты получили ожидаемые verdicts.*

# 1. ЧТО ТАКОЕ POLYMATH-ATLAS

Polymath-Atlas — попытка построить типизированное операционное пространство для познания, анализа, инженерии, проверки и рефлексии. Центральный переход: не «слово = смысл», а «лексема = кандидат на операционный примитив», после чего задаются тип, вход, выход, контракт, инвариант, механизм, эффекты, неопределённость и допустимая композиция.

*РЕМАРКА: Сильная сторона исходной идеи — переход от словаря терминов к системе допустимых преобразований между состояниями, представлениями, моделями, решениями и действиями.*

Исторический корпус прямо фиксирует, что исходный массив терминов неоднороден: в нём смешаны операции, результаты, свойства, артефакты, методы, дисциплины, отношения и процессы. Поэтому нормализация должна предшествовать формализации.

```
ENTITY / OPERATION / RESULT / PROPERTY / METHOD / ARTIFACT / RELATION / PROCESS / META-OPERATION
                          ↓
                   TYPE + CONTRACT
                          ↓
                  STATE TRANSITION
                          ↓
              COMPOSITION / VALIDATION
                          ↓
                 EXECUTION / REPAIR
```

# 2. ПРОЛОГ: ИСХОДНОЕ СОСТОЯНИЕ И КРИЗИС

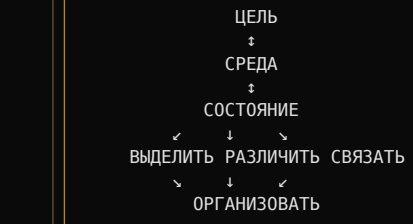
Проект начался с амбициозной попытки формализовать универсальный цикл познания и инженерии через каталог из 418 русских глаголов (типизация, классификация, формализация и т.д.). Однако первичный аудит выявил фундаментальный дефект: смешение в одном списке операций, результатов, свойств, артефактов и дисциплин. Линейная цепочка «Целое → Выделить → Различить → … → Новое Целое» оказалась лишь одним из возможных workflow-паттернов, а не универсальным законом.

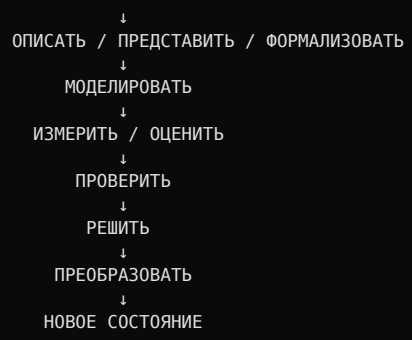
*РЕМАРКА: Отвергнуто: попытка построить операционную алгебру напрямую из естественного языка. Словарь без строгой селекции — свалка. Лингвистический детерминизм порождает этноцентрическую онтологию.*

# 3. КАНОНИЧЕСКАЯ ТРАЕКТОРИЯ И ЕЁ ОГРАНИЧЕНИЕ

```
ЦЕЛОЕ → ВЫДЕЛИТЬ → РАЗЛИЧИТЬ → СВЯЗАТЬ → ОРГАНИЗОВАТЬ → ОПИСАТЬ
→ ПРЕДСТАВИТЬ → ФОРМАЛИЗОВАТЬ → МОДЕЛИРОВАТЬ → КОДИРОВАТЬ → ИЗМЕРИТЬ
→ СОПОСТАВИТЬ → ПРОВЕРИТЬ → ПРЕОБРАЗОВАТЬ → ОПТИМИЗИРОВАТЬ
→ РЕОРГАНИЗОВАТЬ → МЕТАУРОВЕНЬ → НОВОЕ СОСТОЯНИЕ
```

*РЕМАРКА: Эта цепочка сохраняется как каноническая траектория чтения, но не как универсальный алгоритм. Реальные процессы ветвятся, возвращаются, прерываются, откатываются, меняют цель и работают при неполных данных.*






# 4. АРХИТЕКТУРНЫЙ СДВИГ — 5 ОРТОГОНАЛЬНЫХ ОСЕЙ (S-1.0)

Вместо одной иерархии L0-L4 введены 5 ортогональных осей:

- Ось A (Категория): Object, Data, Model, Decision, Action, Meta
- Ось B (Роль): Input, Output, Resource, Target, Constraint
- Ось C (Уровень абстракции): L0 (Reality) → L1 (Data) → L2 (Model) → L3 (Decision) → L4 (Action)
- Ось D (Тип): Формальная решётка с правилами IsSubtype(A, B)
- Ось E (Фаза): Created, Available, Consumed, Destroyed

# 5. ОСНОВНЫЕ ОПЕРАЦИОННЫЕ ФАСЕТЫ

## ВЫДЕЛИТЬ

Определить самостоятельный и существенный элемент.

Абстрагировать, обобщать, конкретизировать, идеализировать, специализировать, детализировать, инстанцировать.

## РАЗЛИЧИТЬ

Определить различия, паттерн, категорию, тип или идентичность.

Обнаружить ≠ распознать ≠ категоризировать ≠ типизировать ≠ классифицировать ≠ идентифицировать.

## СВЯЗАТЬ

Определить отношения между элементами.

Сопоставить, связать, соединить, сопрячь, агрегировать, компоновать, интегрировать, координировать, консолидировать, каузировать, коррелировать.

## ОРГАНИЗОВАТЬ

Построить конфигурацию из элементов и отношений.

Разделить, упорядочить, структурировать; декомпозиция ≠ структурирование.

## ОПИСАТЬ / ФОРМАЛИЗОВАТЬ

Зафиксировать свойства и затем перевести утверждение в формальный язык.

Описание сообщает; спецификация задаёт требования; формализация задаёт язык и правила.

## МОДЕЛИРОВАТЬ

Построить заместитель, сохраняющий релевантные отношения.

Модель не обязана быть полной изоморфной копией.

## ИЗМЕРИТЬ / ПРОВЕРИТЬ

Получить свидетельство и проверить его относительно правила, спецификации или цели.

Верификация ≠ валидация; диагностика не обязана выдавать единственную причину.

## ПРЕОБРАЗОВАТЬ / ОПТИМИЗИРОВАТЬ

Изменить состояние при заданном инварианте; оптимизация требует пространства решений, цели, ограничений и критерия.

Улучшение само по себе не является оптимизацией.

## МЕТАУРОВЕНЬ

Операции над операциями, критериями, моделями и целями.

Рефлексия, аудит, метаоценка, переопределение, рефрейминг, обучение.

*РЕМАРКА: Исходный корпус специально сохраняет различия между близкими терминами; механическое сведение синонимов разрушает различия механизмов и инвариантов.*

# 6. OSA-3.x → ПЕРЕХОД К FORMAL CORE

В исходном аудите зафиксирован набор дефектов, которые нельзя было закрывать локальными строковыми патчами. Среди них: отсутствие доказуемой типовой системы, логики предикатов, модели времени/ресурсов, параллелизма и ветвлений, векторных эффектов, агрегации неопределённости, полноценного Repair Planner, теории идентичности и формального доказательства.

| Дефект | Почему критичен |
|---|---|
| **Типовая система** | IsSubtype не доказан; точечные добавления наследования → бесконечный patching. |
| **Preconditions** | Строки вроде DataSufficiency >= threshold не являются логическим выводом. |
| **Destroys** | Неясно, уничтожается ли объект, факт, свойство или состояние. |
| **Uncertainty** | «inherit» без математического определения — placeholder, а не контракт. |
| **Invariant Engine** | Проверка пересечения required/destroyed — sanity check, а не теория инвариантов. |
| **Testing** | 15 негативных кейсов не доказывают способность системы корректно проходить валидные графы. |
| **Formal Verification** | Lean/Coq проверяет код относительно спецификации; он не делает саму спецификацию истинной. |
| **Repair** | BFS по операторам без формальных контрактов — патчер, а не доказательный planner. |
| **Identity** | RESTORE и RECREATE требуют различения identity и lineage. |
| **Goal validation** | Ценность цели не выводится внутренней формальной системой без внешнего критерия. |

*РЕМАРКА: Этот список не является исторической мелочью: именно эти дефекты определяют, почему OSA-4.0 должен начинаться с семантики, а не с JSON Schema.*

*РЕМАРКА: Отвергнуто: destroyed ∈ state ⇒ violation. Оператор DELETE должен уничтожать данные. Ошибка возникает только если уничтоженный ресурс нужен достижимому продолжению графа (Liveness Analysis).*

# 7. OSA-4.0 — FORMAL CORE / SEMANTIC KERNEL v0.1

Первый формальный слой разводит четыре измерения сущности: EntityCategory, Role, AbstractionLevel и EpistemicStatus; затем вводит TypeLattice, Predicate, EvalResult, State и Operator.

```
data ArtifactType = {
  category :: EntityCategory,
  role :: Role,
  level :: AbstractionLevel,
  epistemics :: EpistemicStatus,
  typeTag :: String
}
isSubtype :: TypeLattice -> ArtifactType -> ArtifactType -> Bool
Predicate =
    FactExists String
  | Not Predicate
  | And Predicate Predicate
  | Or Predicate Predicate
  | Implies Predicate Predicate
  | Gte StateVariable Value
  | Eq StateVariable Value
  | ForAll String Predicate
  | Exists String Predicate
EvalResult = Proven True | Proven False | Unknown
```

*РЕМАРКА: Главный прорыв v0.1: предусловие становится вычисляемым предикатом, а недостаток данных получает отдельный статус Unknown вместо ложного Reject.*

```
State = {
  facts, objects, resources, permissions,
  provenance, uncertainty, temporalValidity
}
Operator = {
  inputType, outputType, preconditions, postconditions,
  effectVector, uncertaintyTransfer, destroys, produces
}
```

# 8. SEMANTIC EXECUTION KERNEL v0.2

v0.2 вводит Fact с provenance, epistemic status, validity и owner; Requirement для Liveness; OperatorEffect с produces/consumes/preserves/invalidates/modifies/requires; TriBool; goal-relative liveness; branch-aware state intersection; constrained Repair Planner; verdict semantics.

```
Fact = { factId, value, provenance, epistemic, validity, owner }
OperatorEffect = { produces, consumes, preserves, invalidates, modifies, requires }
TriBool = True | False | Unknown

liveSet(graph, node):
  GoalNode → requirements(graph)
  OpNode op → op.requires ∪ (liveAfter \ op.produces)
```

```
checkLiveness(graph, node):
  invalidated ∩ liveAfter = ∅  ⇒ legal

Verdict:
  any FALSE                          → REJECT
  all TRUE                           → PASS
  valid repair with no Unknown       → REPAIR
  repair exists but has Unknown      → UNDECIDABLE
  otherwise                          → REJECT
```

*РЕМАРКА: Здесь возникла последующая проблема: set subtraction достаточен для атомарных FactID, но недостаточен для логических предикатов. Это приводит непосредственно к WP в v0.3.1.*

# 9. SEMANTIC KERNEL v0.3 — PREDICATE & LATTICE ALGEBRA

v0.3 заменяет FactID-level Liveness на Predicate-level Liveness и вводит эпистемическую решётку с TRUE_KNOWN, FALSE_KNOWN, UNKNOWN, ABSENT и CONFLICT.

```
Requirement = { predicate :: Predicate, criticality :: Blocker | Soft }
LiveBefore(Op, LiveAfter) = Op.Requires ∪ (LiveAfter \ Op.Produces)
Safety:
  Op.Invalidates ∩ LiveAfter = ∅
  Op.Consumes ∩ LiveAfter = ∅

EpistemicValue = TRUE_KNOWN | FALSE_KNOWN | UNKNOWN | ABSENT | CONFLICT
TRUE_KNOWN ⊔ FALSE_KNOWN = CONFLICT
TRUE_KNOWN ⊔ UNKNOWN     = UNKNOWN
TRUE_KNOWN ⊔ ABSENT      = UNKNOWN

Repair objective:
V = [SafetyViolations, ResidualProofObligations,
     EffectPolicyViolations, ResidualUncertainty, ExecutionCost]
Lexicographic minimization: Safety dominates all later components.
```

*РЕМАРКА: Эта версия была сильнее предыдущей, но последующий аудит показал, что ABSENT нельзя просто считать ещё одним истинностным значением без отдельной семантики presence.*

# 10. SEMANTIC TORTURE SET T01–T15 — ПЕРВИЧНЫЙ ПРОГОН

| ID | Сценарий | Вердикт | РЕМАРКА / ИСПРАВЛЕНИЕ |
|---|---|---|---|
| T01 | MEASURE → CALIBRATE → MODEL | Первоначально заявлен PASS. | Дыра: DataSufficiency не было явно произведено. Исправление: сделать его output MEASURE или derived predicate. После фикса PASS только при доказанной достаточности. |
| T02 | CREATE X → CREATE Y → DELETE X → ANALYZE Y | PASS. | Корректная проверка независимого уничтожения. |
| T03 | CREATE X → DELETE X → ANALYZE X | REJECT / E_LIVENESS_VIOLATION. | X принадлежит LIVE_OUT DELETE. |
| T04 | DELETE X → RESTORE X → ANALYZE X | Первоначально REJECT. | Нужно различить resurrection same-ID и recreation new-ID. Это два теста: T04a/T04b. |
| T05 | MEASURE → MODEL, Sufficiency UNKNOWN | UNDECIDABLE. | UNKNOWN не равен FALSE_KNOWN. |
| T06 | MEASURE → MODEL, Sufficiency 0.2 < 0.8 | REJECT. | Предусловие доказанно ложно. |
| T07 | MEASURE → TELEPORT → MODEL | REJECT / E_UNKNOWN_OPERATOR. | Оператор отсутствует в Registry. |
| T08 | Branch A: X exists; Branch B: X absent → Merge | UNDECIDABLE. | TRUE_KNOWN ⊔ UNKNOWN → UNKNOWN после правильного разделения presence и truth. |
| T09 | Branch A: TRUE(X); Branch B: FALSE(X) → Merge | REJECT / CONFLICT. | Conflict нельзя понижать до Unknown. |
| T10 | Branch A: X=10; Branch B: X=11 → Merge | UNDECIDABLE для точного значения. | Существование и значение — разные компоненты состояния. |
| T11 | CALIBRATE X → MODIFY X → USE X (USE requires Calibrated) | REJECT. | MODIFY инвалидирует предикат, который ещё live. |
| T12 | CREATE X → CONSUME X → CONSUME X | REJECT / E_INVARIANT_VIOLATION. | Второе consume недопустимо. |
| T13 | DECIDE → EXECUTE без authorization | Первоначально ошибочно привязан к LAW-008. | Нужен отдельный LAW-011 Effect Authorization Soundness. |
| T14 | MEASURE → MODEL UNKNOWN; repair CALIBRATE, standard UNKNOWN | UNDECIDABLE. | Repair существует как гипотеза, но proof obligation остаётся. |
| T15 | Cheap unsafe vs safe expensive repair | Первоначальная запись содержала перевёрнутые Safety bits. | Правильно: unsafe=[1,...,1], safe=[0,...,3]. Выбирается safe path. |

*РЕМАРКА: Итог: 15/15 нельзя объявлять доказательством soundness. Эти тесты выявили скрытые недоопределённости. Это именно то, для чего Torture Set и нужен.*

# 11. SEMANTIC KERNEL v0.3.1 / v0.4 — ТЕКУЩАЯ БАЗА (ALGEBRAIC CLOSURE)

v0.3.1/v0.4 фиксирует пять независимых пространств: Type, State, Effect, Epistemic и Predicate. Ключевой переход — PredicateSpace становится отдельным математическим объектом.

```
T = TypeSpace
S = StateSpace
E = EffectSpace
U = EpistemicSpace
P = PredicateSpace
Operator execution:  Op : S × T → S' × T' × E
Predicate semantics: ⟦P⟧_S → PredicateTruth
```

ABSENT больше не является истинностным значением предиката. Presence и PredicateTruth разводятся.

```
ResourcePresence = Present | Absent
PredicateTruth   = TRUE_KNOWN | FALSE_KNOWN | UNKNOWN | CONFLICT

⟦P⟧_S → PredicateTruth
DataExists(X) = FALSE_KNOWN  только при наличии доказательства несуществования X.
Если X просто отсутствует из state-map: DataExists(X) = UNKNOWN.
```

Epistemic Join становится полной таблицей и проверяется на idempotence, commutativity и associativity.

| ⊔ | TRUE | FALSE | UNKNOWN | ABSENT | CONFLICT |
|---|---|---|---|---|---|
| **TRUE** | TRUE | CONFLICT | UNKNOWN | UNKNOWN | CONFLICT |
| **FALSE** | CONFLICT | FALSE | UNKNOWN | UNKNOWN | CONFLICT |
| **UNKNOWN** | UNKNOWN | UNKNOWN | UNKNOWN | UNKNOWN | CONFLICT |
| **ABSENT** | UNKNOWN | UNKNOWN | UNKNOWN | ABSENT | CONFLICT |
| **CONFLICT** | CONFLICT | CONFLICT | CONFLICT | CONFLICT | CONFLICT |

*РЕМАРКА: Это уже настоящая join-semilattice для PredicateTruth. ABSENT намеренно вынесен из неё. ABSENT (объекта нет в ветви) ≠ FALSE (доказано, что предикат ложен).*

Главный фикс Liveness: вместо set subtraction вводится weakest-precondition transformer.

```
WP(Op,Q):
  S ⊨ WP(Op,Q)  iff  Exec(Op,S) ↓ S'  and  S' ⊨ Q
LiveBefore(Op,Q) = Pre(Op) ∧ WP(Op,Q)

WP учитывает produces, consumes, invalidates, modifies,
logical implication, substitution, aliasing и resource identity.
```

*РЕМАРКА: WP закрывает дыру, которую простое множество FactID не могло закрыть.*

MODIFY обязан явно декларировать инвалидируемые предикаты; invalidate не означает автоматически доказать отрицание предиката.

```
MODIFY:
  Modifies = {X}
  Invalidates = {Calibrated(X), Verified(X)}
  Preserves = {Exists(X), Owner(X)}
Invalidates(P) ≠ proves ¬P
```

Identity/alias semantics различают resurrection и recreation.

```
Resurrection (ЗАПРЕЩЁН):
  X_old: Destroyed → X_old: Available   same identity  ⇒ REJECT

Recreation (РАЗРЕШЁН):
  X_old: Destroyed
  RESTORE/RECREATE → X_new: Available
  ID(X_new) ≠ ID(X_old) + Provenance   ⇒ potentially legal
```

Effect Authorization выводится в отдельный закон. Effect Monotonicity отвечает только за накопление эффектов; authorization — отдельное ограничение исполнения.

```
LAW-011:
  RequiredAuth(Op) = AuthFor(Op.Effect)
  Exec(Op,S) ⇒ Authorized(S,Op)
  FALSE_KNOWN → REJECT
  UNKNOWN     → UNDECIDABLE
```

Lexicographic Repair Optimizer:

```
V = [SafetyViolations, ResidualProofObligations, EffectPolicy, Uncertainty, Cost]
V1 = [1, 0, 0, 0, 1]  (Unsafe, Cheap)
V2 = [0, 0, 0, 0, 3]  (Safe, Expensive)
Выбор: V2  (т.к. Safety доминирует)
```

# 12. LAWBOOK — АЛГЕБРАИЧЕСКИЕ ЗАКОНЫ

| ID | Закон | Формула / правило | РЕМАРКА |
|---|---|---|---|
| LAW-001 | Type Reflexivity | ∀A: A ⊑ A | Базовое свойство отношения подтипизации. |
| LAW-002 | Type Transitivity | A ⊑ B ∧ B ⊑ C ⇒ A ⊑ C | Запрещает локальную несогласованность цепочек типов. |
| LAW-003 | State Conservation | Если Op не изменяет X, State_after(X)=State_before(X). | Оператор не может неявно менять компоненты состояния. |
| LAW-004 | No Resurrection | Destroyed/Invalid не переходят в Available с тем же identity. | Recreate с новым ID не является resurrection. |
| LAW-005 | Consume Safety | consume(Available)=Consumed; повторный consume недопустим. | Разводит потребление и уничтожение. |
| LAW-006 | Destroy Safety | KILL(X) допустим ⇔ X ∉ LIVE_OUT. | Безопасность зависит от будущей необходимости. |
| LAW-007 | Liveness Soundness | Проверенный граф не должен иметь неразрешимых live requirements. | Liveness должен быть доказан относительно целей. |
| LAW-008 | Effect Monotonicity | Effect(G)=⊕ Effect(n), затем сравнение с policy. | Не смешивать с authorization. |
| LAW-009 | Epistemic Non-Overclaim | U_out < U_in требует явного evidence/validation. | Нельзя уменьшать неопределённость декларативно. |
| LAW-010 | Composition Soundness | Существует S' с валидным переходом и Preconditions(B). | Композиция — существование корректного промежуточного состояния. |
| LAW-011 | Effect Authorization Soundness | Exec(Op,S) ⇒ Authorized(S,Op). | Отдельный policy law. |
| LAW-012 | Predicate Transformer Soundness | S ⊨ WP(Op,Q) ⇒ Exec(Op,S)=S' ∧ S'⊨Q. | Фундаментальная семантика WP. |
| LAW-013 | Join Commutativity | x⊔y=y⊔x | Branch merge не зависит от порядка. |
| LAW-014 | Join Associativity | (x⊔y)⊔z=x⊔(y⊔z) | Многостороннее слияние должно быть однозначным. |
| LAW-015 | Join Idempotence | x⊔x=x | Повтор одной ветви не должен менять результат. |
| LAW-016 | Recreation ≠ Resurrection | ID_new≠ID_old допускает новый объект при корректном provenance. | Различие identity и lineage. |
| LAW-017 | Predicate Invalidation Soundness | Invalidates(P) делает старое доказательство P недействительным. | Это не обязательно доказывает ¬P. |
| LAW-018 | Repair Soundness | Valid repair должен быть type/contract/invariant/auth/WP-safe. | Планировщик ищет доказуемый план, а не один удачный сценарий. |

# 13. TORTURE SET v0.4 — ПРАВИЛЬНЫЙ ПРОТОКОЛ

Следующая проверка не должна быть механическим расширением T01–T15. Для каждого закона нужен минимальный набор контрпримеров: positive, negative, boundary, branch и repair.

```
LAW
 ↓
POSITIVE
NEGATIVE
BOUNDARY
BRANCH
REPAIR
 ↓
MINIMAL BASIS SET
 ↓
MODEL CHECK
 ↓
KERNEL v0.4
```

*РЕМАРКА: При 18 законах это потенциально 90 проверочных позиций, но не обязательно 90 уникальных графов. Один минимальный контрпример может одновременно проверять несколько законов.*

# 14. POLYMATH-ATLAS v5 — ENGINEERING PIPELINE

Архитектурный слой v5 решает уже другую проблему: как масштабировать 301+ узлов, не превращая LLM в владельца графа. Каноническая схема:

```
PLAN → RESERVE → CONTEXT-PACK → GENERATE → PARSE
                                    ↓
                        STRUCTURAL VALIDATION
                                    ↓
                    SEMANTIC REVIEW / SYNTHESIS
                                    ↓
                            HUMAN GATE
                                    ↓
                        COMMIT → INDEX
```

*РЕМАРКА: Ключевой принцип: LLM предлагает содержание, но не управляет графом. ID, ссылки, топология и принятие должны контролироваться машинным слоем.*

Шесть слоёв: спецификация проекта; реестр графа; планировщик заданий; генерационный контекст; prompt compiler; нормализация/валидация/публикация.

```
polymath-atlas/
├── atlas-spec/
│   ├── ontology.yaml
│   ├── edge-rules.yaml
│   ├── glossary.yaml
│   └── node-schemas/
├── registry/
│   ├── nodes.yaml
│   ├── edges.yaml
│   ├── id-registry.yaml
│   └── build-state.yaml
├── tasks/  bundles/  context/  prompts/
├── steps/  concepts/  invariants/  mechanisms/
├── reports/  scripts/
```

Жизненный цикл узла: planned → reserved → generated → parsed → validated → reviewed → accepted / rejected. Стабильный logical key отделяется от display ID.

```
stable_key = lateral:STEP-002:types-of-growth
display_id = STEP-042
```

Валидационные ворота: формат → схема → ID/ссылки → топология → локальная семантика → терминология → человеческий просмотр.

*РЕМАРКА: Структурная ошибка, графовая ошибка, топологическая ошибка, семантическая ошибка и редакционная ошибка не должны обрабатываться одинаково повторным обращением к LLM.*

# 15. МАСШТАБ 301 ШАГА И СТРУКТУРА ГРАФА

В проектном корпусе зафиксирована модель с 301 шагом, отдельными concepts/invariants/mechanisms и автоматическим индексом. Markdown рассматривается как человекочитаемый рендер структурированных данных, а не как единственный источник истины.

```
graph/
  ATLAS-INDEX.yaml
  ATLAS-GRAPH.yaml
  ATLAS-STATS.json
  steps/
    STEP-001.md ... STEP-301.md
concepts/  invariants/  mechanisms/  generator/
```

*РЕМАРКА: При 301 шаге полный граф нельзя постоянно подавать LLM. Нужен локальный context pack + компактный глобальный индекс; полные тексты подставляются только для релевантных узлов.*

# 16. ОТВЕРГНУТЫЕ АЛЬТЕРНАТИВЫ — КАНОН РЕШЕНИЙ

| ID | Альтернатива | Почему НЕ ИДЁТ |
|---|---|---|
| A1 | «Словарь для полноты» | Словарь без селекции превращается в свалку. |
| A2 | «Типизировать постфактум» | Типизация сортирует уже созданную свалку, но не задаёт критерий. |
| A3 | «Запретить взаимозаменяемость» | Контекстозависимость делает статический запрет чрезмерным. |
| A4 | «Заменить цикл каталогом» | Каталог без routing — справочник, а не операционная система. |
| A5 | «Разделить только по типу выхода» | Один оператор может иметь разные выходы в разных контекстах. |
| A6 | «Инженерный компромисс» | Trade-off должен иметь формальный анализ, а не быть риторическим словом. |
| A7 | «DECIDE = логика, AUTHORIZE = санкция» | Санкция без контракта остаётся пустой абстракцией. |
| A8 | «Bridge добавим позже» | BFS по невалидированным операторам создаёт комбинаторный взрыв. |
| A9 | «Расширить до SAT-решателя» | Строки JSON не становятся формальными предикатами от одного SAT-ярлыка. |
| A10 | «Destroys = только физические данные» | PreviousState может быть данными или состоянием; граница требует формальной семантики. |
| A11 | «Uncertainty потом» | Если поле уже присутствует в схеме, оно обязано иметь semantics. |
| A12 | «Инварианты через annotations» | Annotation — комментарий, а не механизм доказательства. |
| A13 | «Позитивные кейсы тривиальны» | Для заявленной состоятельности нужен хотя бы доказанный позитивный проход. |
| A14 | «Lean/Coq всё докажет» | Формальная верификация проверяет код относительно спецификации, но не истинность спецификации. |
| A15 | «Fuzzing всё найдёт» | Случайные графы часто тривиально некорректны и плохо покрывают семантические границы. |
| A16 | «Исправить expected» | Подгонка ожидаемого результата под реализацию не исправляет спецификацию. |
| A17 | «Поменять порядок проверок» | Порядок проверок — семантическое решение и требует обоснования. |
| A18 | «Bootstrapping решает circularity» | Самопроверка не становится внешним oracle только от нового имени. |
| A19 | «Нелинейность = метауровень» | Метауровень без операциональной семантики остаётся декларацией. |
| A20 | «Практика ограничивает мета» | Практика не является формальным критерием сама по себе. |

# 17. ЭПИЛОГ: ОТКРЫТЫЕ ПРОБЛЕМЫ И ГРАНИЦЫ

Полиматический аудит признает границы текущей формализации. Следующие концепты не могут быть полностью формализованы в рамках ядра и требуют внешнего оракула или расширения системы:

- Интерпретация: Контекст бесконечен, смысл не типизируется. Оставлена как сквозной комбинатор.
- Модель Агента: Компетенция и юрисдикция — социальные конструкты, не поддающиеся строгой алгебре.
- Валидация Цели: Формальная система не может оценивать этичность или истинность самой цели.

# 18. ФИНАЛЬНАЯ ТРАЕКТОРИЯ

```
OSA-3.x
   ↓
AUDIT / DEFECT CATALOG
   ↓
FORMAL CORE v0.1
   ↓
SEMANTIC EXECUTION KERNEL v0.2
   ↓
PREDICATE & LATTICE KERNEL v0.3
   ↓
TORTURE SET T01–T15
   ↓
COUNTEREXAMPLES REVEAL SEMANTIC GAPS
   ↓
KERNEL v0.3.1 / v0.4
   ├── PredicateSpace
   ├── WP / weakest preconditions
   ├── Presence ≠ Truth
   ├── complete Join Semilattice
   ├── Recreation ≠ Resurrection
   ├── Effect Authorization
   ├── Predicate Invalidation
   ├── Alias / Identity
   └── Repair Soundness
   ↓
LAWBOOK (18 axioms)
   ↓
MINIMAL BASIS TORTURE SET
   ↓
MODEL CHECK
   ↓
KERNEL v0.4 (closed)
   ↓
ONLY THEN
Atlas S-1.0 Meta-Schema
```

Главный архитектурный принцип: Meta-Schema должна быть сериализацией уже определённой математики, а не местом, где математика впервые появляется.

*РЕМАРКА: Это и есть граница между фасадом и фундаментом. JSON/YAML/Lean выбираются после того, как определены переходы состояний, логика предикатов, join, liveness, effects, authorization, identity и repair semantics.*

*РЕМАРКА: Финальный статус: Polymath-Atlas — это не универсальный алгоритм мышления и не готовое ядро AGI. Это строгая, типизированная операционная семантика, способная строить, проверять и аудировать графы преобразований, честно фиксируя собственные эпистемические границы.*

Compiled by Polymath-Architect • OSA-4.0 Full Unified Codex • 08 August 2026